

Immersive Visualization of Open Geospatial Data in Unreal Engine

Tristan King^{*} Kyle Davis[†] Bradley Saunders[‡] Ryan Zuber[§] Priya Rajasagi^{||}
Christina Lukaszczyk^{||} Anita Komlodi^{**} Lee Boot^{††}

Imaging Research Center
University of Maryland, Baltimore County

ABSTRACT

Immersive geospatial data visualizations are a growing area of interest for research and commercial efforts. However, access to the necessary technologies is often limited by the expense of proprietary solutions. Furthermore, available solutions do not offer the ability to select and render only portions of global maps, creating significant and unnecessary inefficiencies. We developed a pipeline for efficient generation, rendering, and interaction of open-source geospatial vector features in Unreal Engine™ Virtual Reality (VR) at almost any scale, and a user interface for interacting with these vector map features and georegistered data visualizations (as game objects) in VR applications.

Index Terms: Human-centered computing—Visualization—Visualization techniques—; Human-centered computing—Visualization—Virtual Reality

1 INTRODUCTION

Geospatial data visualizations, or geovisualizations, have been used across wide disciplinary areas of interest, including education [3], public health [1], and urban planning. Combined with the increasing accessibility of virtual reality (VR) and mixed reality (XR) hardware platforms, geospatial visualizations are poised to become fundamental aspects of the next age of computing. To best serve public policy and planning, we sought to improve the design and reduce the cost of geovisualization compared to currently available commercial offerings using open-source tools to build a solution in Unreal Engine. Related studies, such as the work by Höhl [2], still use commercial tools to function. Our goal was to make a tool solely reliant on open-source building blocks in addition to our proprietary work. The capacity to include and juxtapose multiple data types is a valuable feature in geovisualization [4]. Consequently, we developed the capacity to represent multiple, abstract data types to establish a useful, high-performing platform that will ultimately enable the integration of many datasets, data types, visualization modes, and representations.

2 IMPLEMENTATION

2.1 Geovisualization

We begin by processing a user-defined shapefile (SHP) using an open-source SHP parser (<https://github.com/gwentruong/utshp>) modified to fit Unreal Engine's coordinate system. This outputs a list of

records, each record being a complete polygon. We then triangulate each record using an open-source variation of the ear-clipping triangulation algorithm, Earcut (<https://github.com/mapbox/earcut.hpp>). As each record is independently defined, we can process this triangulation in a multithreaded manner. Additionally, we combine each triangulated record into a single array, enabling us to render the entire vector feature in a single draw call within Unreal Engine. Finally, we apply the Web Mercator projection to all vertices of the feature to accurately geocode the map within Unreal Engine's coordinate system. See Fig. 1

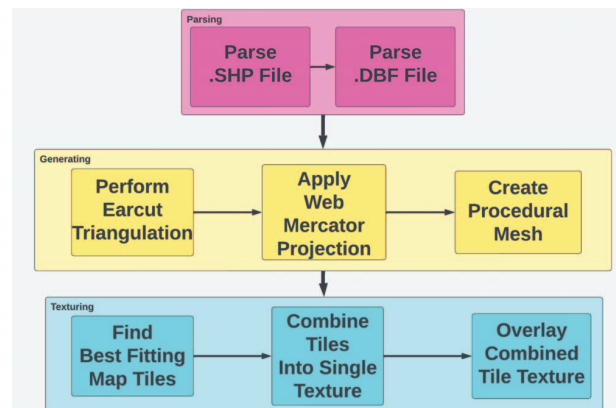


Figure 1: A flowchart of the entire visualization and map generation pipeline, with three main steps: Parsing, Generating, and Texturing

In order to render the processed shapefile, we use the open-source plugin Runtime Mesh Component (<https://github.com/TriAxis-Games/RuntimeMeshComponent>). This component renders a dynamic mesh object (in this case, our map) much faster than the default Procedural Mesh Component shipped in Unreal Engine. We then position the dynamic mesh object on a virtual table within a scene, clip the map to the table's edges, and determine, load, and overlay the best-fitting OpenStreetMap tiles on our dynamic mesh object. These tiles are determined based on the current scale and position of the mesh within the "viewport" of the virtual table object—converting these to tile coordinates and zoom levels accordingly.

The combination of open-source map tiles and dynamic, user-determined vector features allows us to render almost any level of geographic detail, from national to street level. Without the need for extensive modifications, a user can display almost any combination of vector features, overlay map tiles, and transform the features to display at all scales from a tabletop to the real world. See Fig. 2

2.2 User Interactions

Platform interaction methods were designed for novice VR users who preferred menu items over controller buttons in our initial

^{*}tking5@umbc.edu

[†]kdavis18@umbc.edu

[‡]bsaunde1@umbc.edu

[§]zuj1@umbc.edu

^{||}u17@umbc.edu

^{||}clukasz1@umbc.edu

^{**}komlodi@umbc.edu

^{††}boot@umbc.edu

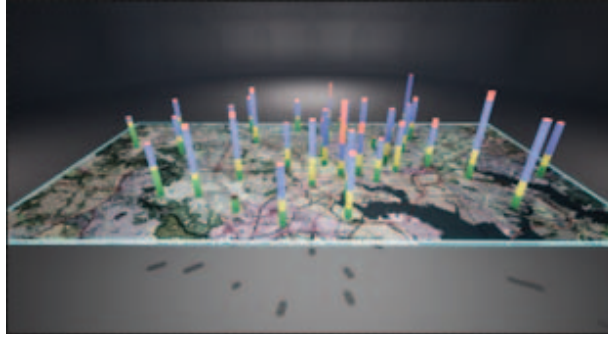


Figure 2: A view of the map zoomed to view street and building level details, with tabletop clipping applied to map edges and localized data overlay

round of internal testing. Users interact with the map generation pipeline using custom-built menus affixed to one of the XR controllers. This allows users to translate, zoom, or scale the map with sliders and joysticks. Beams affixed to the front of controllers act as pointers guiding menu and map selection. For example, users can select an area of the map with their controller pointers and extract corresponding attributes, such as exact latitude and longitude locations, or regional identifiers, such as zip codes. We created display options—such as toggling map tile visibility and detailed versus simplified rendering of geographic regions and boundaries—available using buttons on the user interface. This combination of controls makes the tool a flexible and extensible platform for georeferenced data visualizations

2.3 Georegistered Data

This platform facilitates the geolocation and overlay of qualitative and quantitative data types and can include other 3D assets, such as buildings and 3D data charts. Leveraging rendering techniques native to Unreal Engine (instanced rendering) facilitates the visualization of large datasets—such as socioeconomic data from the US Census Bureau—using bar charts. Regional boundary colors, heights, and other visual parameters are available to encode additional data. Text billboards and labels have been integrated with the map data to highlight areas of interest, exact geographic identifiers such as zip codes, and make numerical values visible to the user.

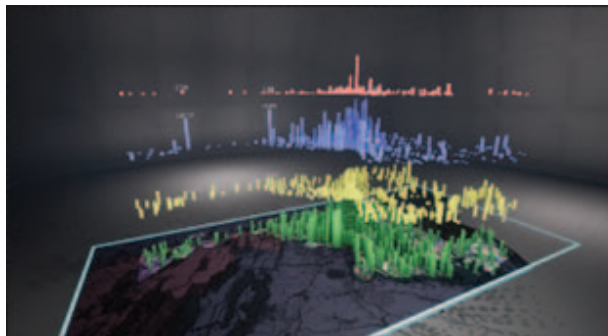


Figure 3: An expanded view of four sets of data bars, for every zip code in the state of Maryland

3 RESULTS AND PERFORMANCE

Using Unreal Engine 4.27.1, we ran an example scene with two vector layers (USA states and Maryland zip codes) and four georegistered data layers on a Windows PC using a Meta Quest Pro headset via an Oculus Link cable. The application runs at an average of 130 fps, which far exceeds the threshold for simulator sickness, ensuring a smooth user experience. Figure 4 shows triangulation and procedural mesh generation metrics for each layer.

Vector Feature	Vertices	Triangles	Triangulation Time	Mesh Generation Time
USA States	971,263	970,791	104.4ms	53.9ms
Maryland Zip Codes	530,620	528,480	27.2ms	28.1ms

4 CONCLUSION

This system could expand affordable access to immersive geographic visualizations for a wide range of use cases. The capacities we have developed are important for promoting XR as a platform for exploring and analyzing geographically referenced data. The platform is flexible, highly-performant, and dynamic. User interactions and interfaces were developed with a focus on intuitive usability and aesthetic design. User testing has established that the platform is useful and highly usable for georeferenced data exploration. The platform will now serve as the foundation for integrating a wide range of data types using multiple modes of visualization and representation in immersive environments, including concept mapping, timeline-driven policy milestones, imagery, video, audio, and text. We expect this to catalyze multimodal, cross-dataset analysis. We are currently developing the following:

- Support for other geospatial features and file types (i.e., GeoJSON, Topography, etc.)
- The ability to save or bookmark states and views of the visualization to easily revisit them across multiple separate sessions or in sequence
- Support for simultaneous multi-user viewing and interaction.

After these features are developed, we will carry out further user studies to evaluate their impact on the overall system.

REFERENCES

- [1] E. Goetschel, J. Sekaran, W. Ren, M. He, N. Ogbonnaya, M. Nkereuwem, I. Mapfunde, C. Martin, C. Cogburn, and S. Feiner. Coviz: Visualization of effects of covid-19 on new york city through socially impactful virtual reality. *2021 IEEE Conference on Virtual Reality and 3D User Interfaces Abstracts and Workshops (VRW), Virtual Reality and 3D User Interfaces Abstracts and Workshops (VRW), 2021 IEEE Conference on, VRW*, pp. 703 – 704, 2021.
- [2] W. Höhl. Official survey data and virtual worlds—designing an integrative and economical open source production pipeline for xr-applications in small and medium-sized enterprises. *Big Data 'I&' Cognitive Computing*, 4(4):1 – 15, 2020.
- [3] A. Philips, A. Walz, A. Bergner, T. Graeff, M. Heistermann, S. Kienzler, O. Korup, T. Lipp, W. Schwanghart, and G. Zeilinger. Immersive 3d geovisualization in higher education. *Journal of Geography in Higher Education*, 39(3):437 – 449, 2015.
- [4] A. Çöltekin, I. Lokka, and M. Zahner. On the usability and usefulness of 3d (geo)visualizations – a focus on virtual reality environments. *International Archives of the Photogrammetry, Remote Sensing 'I&' Spatial Information Sciences*, 41(B2):387 – 392, 2016.